Module 01

FastAPI

Build async REST APIs from zero to production

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. What is a Web Framework?

When a user's browser or app talks to your backend, it sends an HTTP request (think: a letter with an address and a message). Your backend reads it, does some work, and sends back an HTTP response (the reply). A **web framework** is the library that handles all the boring plumbing — reading the request, parsing the URL, routing it to the right function, and sending the response back.

FastAPI is Python's modern async-first web framework. It is built on top of **Starlette** (the ASGI server layer) and **Pydantic** (data validation). Compared to Flask it is faster and has built-in type safety. Compared to Django it is lighter and requires far less boilerplate.

2. Install & First Endpoint

```
# Install (inside your venv)
pip install fastapi uvicorn

# hello.py
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Hello, world!"}

# Run it:
# uvicorn hello:app --reload --port 8000
# Visit http://localhost:8000  →  {"message": "Hello, world!"}
# Docs:  http://localhost:8000/docs  (auto-generated Swagger UI)
```

> ***Note:*** *FastAPI automatically generates interactive API documentation at /docs — you get this for free with zero extra configuration.*

3. HTTP Methods and Path Parameters

REST APIs follow a convention: GET = read, POST = create, PUT/PATCH = update, DELETE = remove.

```
from fastapi import FastAPI

app = FastAPI()

# GET with path parameter
@app.get("/items/{item_id}")
def get_item(item_id: int):        # FastAPI reads {item_id} from URL and casts to int
    return {"id": item_id, "name": f"Item {item_id}"}

# GET with query parameters
@app.get("/search")
def search(q: str, limit: int = 10):   # ?q=hello&limit=5
    return {"query": q, "limit": limit, "results": []}
# POST with a body
```

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    price: float

@app.post("/items/")
def create_item(item: Item):    # FastAPI reads JSON body → Item object
    return {"created": item.name, "price": item.price}
```

4. Status Codes and Responses

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

fake_db = {1: "Alice", 2: "Bob"}

@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id not in fake_db:
        raise HTTPException(status_code=404, detail="User not found")
    return {"name": fake_db[user_id]}

# FastAPI maps Python exceptions to HTTP status codes automatically.
```

5. Routers — Splitting Your API into Modules

As your API grows, one file becomes messy. FastAPI's **APIRouter** lets you split routes into separate files and include them in the main app — exactly like Flask Blueprints.

```
# routers/users.py
from fastapi import APIRouter

router = APIRouter(prefix="/users", tags=["users"])

@router.get("/")
def list_users():
    return []

@router.post("/")
def create_user():
    return {"created": True}

# ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# main.py
from fastapi import FastAPI
from routers.users import router as users_router

app = FastAPI()
app.include_router(users_router)   # all /users/* endpoints registered
```

6. Dependency Injection (Depends)

Dependency injection lets you write reusable logic (auth checks, DB sessions, settings) that FastAPI automatically calls and injects into your route functions.

```
from fastapi import FastAPI, Depends, HTTPException, Header

app = FastAPI()

# ■■ Dependency: extract and verify API key ■■
def verify_api_key(x_api_key: str = Header(...)):
    if x_api_key != "secret-key-123":
        raise HTTPException(status_code=401, detail="Invalid API key")

# ■■ Apply to one route ■■
@app.get("/protected", dependencies=[Depends(verify_api_key)])
def protected_route():
    return {"data": "secret data"}

# ■■ Apply to ALL routes on a router ■■
from fastapi import APIRouter
secure_router = APIRouter(dependencies=[Depends(verify_api_key)])

@secure_router.get("/admin")
def admin_route():
    return {"admin": True}
```

> ***Note:*** *In ContentAutomation2 every router except /unsubscribe is protected with verify_api_key via Depends — see backend/app/api/main.py:63.*

7. Middleware and CORS

Middleware is code that runs on every request/response, before and after your route handlers. CORS (Cross-Origin Resource Sharing) middleware tells browsers whether your frontend (e.g. localhost:3000) is allowed to call your backend API.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:3000"],  # frontend origin
    allow_credentials=False,
    allow_methods=["*"],    # GET, POST, PUT, DELETE, etc.
    allow_headers=["*"],
)

# Without this, browsers block fetch() calls from your Next.js app.
```

8. Lifespan — Startup & Shutdown Logic

The **lifespan** context manager runs code when the server starts and when it shuts down. Use it to open database connections, Redis pools, or load ML models that you want to share across all

requests.

```
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    # ■■ startup ■■
    app.state.db = await connect_database()
    print("DB connected")

    yield          # server is running — handle requests here

    # ■■ shutdown ■■
    await app.state.db.close()
    print("DB closed")

app = FastAPI(lifespan=lifespan)

@app.get("/items")
async def items(request: Request):
    db = request.app.state.db   # shared DB connection
    ...
```

> ***Note:*** *ContentAutomation2 uses this to create and close the arq Redis pool at startup/shutdown (backend/app/api/main.py:18-38).*

9. Async Endpoints

FastAPI supports both sync (def) and async (async def) handlers. For I/O-bound work (database queries, external API calls) always use async def. For CPU-bound work use sync functions or run them in a thread pool.

```
import httpx
from fastapi import FastAPI

app = FastAPI()

@app.get("/weather")
async def get_weather(city: str):
    async with httpx.AsyncClient() as client:
        resp = await client.get(f"https://api.weather.com/{city}")
        data = resp.json()
    return {"city": city, "temp": data["temp"]}

# Notice: no blocking — the event loop handles other requests
# while we wait for the weather API to respond.
```

10. How ContentAutomation2 Uses FastAPI

The backend has one FastAPI app created in backend/app/api/main.py. It is split across 7 routers:

- ideas — Gate 1 approval (approve/reject content ideas)

- drafts — Gate 2 approval (approve/reject content drafts)
- triggers — trigger research/scoring/creation agents
- tables — read-only views of every DB table for the admin panel
- subscribers — unsubscribe endpoint (public, no auth required)
- knowledge_base — upload and list KB files
- orchestrator — streaming WebSocket chat with the AI orchestrator

11. Practice Exercises

Build these in order — each one teaches a new FastAPI concept:

- Exercise 1: Create a FastAPI app with a GET /hello/{name} that returns 'Hello, {name}!'
- Exercise 2: Add a POST /items endpoint that accepts {name: str, price: float} and returns the item
- Exercise 3: Add a verify_api_key dependency and apply it to POST /items
- Exercise 4: Split GET /items and POST /items into a separate router in items.py
- Exercise 5: Add CORS middleware allowing http://localhost:3000
- Exercise 6: Add a lifespan that prints 'Server starting' and 'Server stopped'
- Exercise 7: Add an async GET /fetch that calls any public API using httpx and returns results